

QUIZ

What is the difference between decision trees with **bagging** and the **random forest** decision trees?

REVIEW OF THE LAST PART OF PREVIOUS LECTURE

Naive Bayes Classifier

Select class with max probability for item X :

$$\mathit{arg} \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)}]$$

REVIEW OF THE LAST PART OF PREVIOUS LECTURE

Naive Bayes Classifier

Select class with max probability for item X :

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)}]$$

Denominator same for all classes: $p(y = c_i | X) \propto p(y = c_i)p(X | y = c_i)$

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i)p(X | y = c_i)]$$

REVIEW OF THE LAST PART OF PREVIOUS LECTURE

Naive Bayes Classifier

Select class with max probability for item X :

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)} \right]$$

Denominator same for all classes: $p(y = c_i | X) \propto p(y = c_i)p(X | y = c_i)$

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[p(y = c_i)p(X | y = c_i) \right]$$

To compute easily assume features independent given target class:
(Naive Bayes)

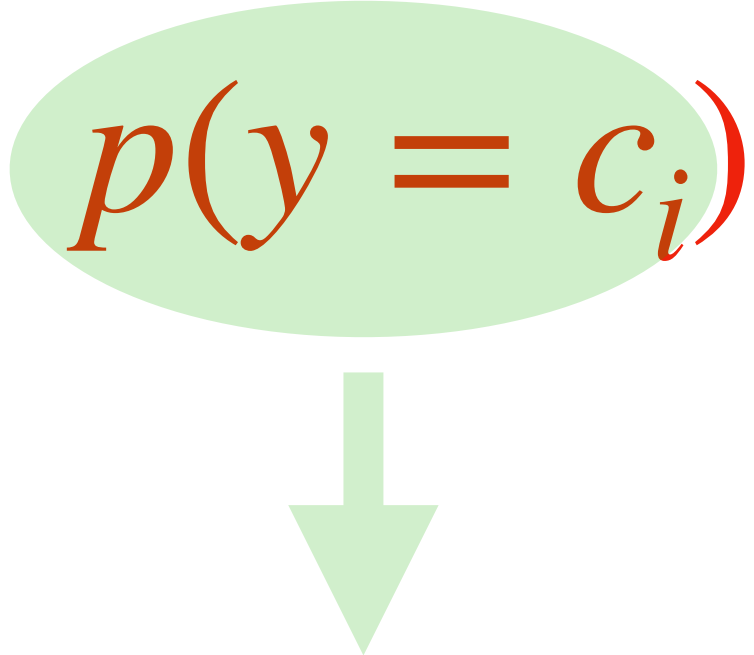
$$p(X | y = c_i) = p(x_1, x_2, \dots, x_m | y = c_i) = \prod_{j=1}^m p(x_j | y = c_i)$$

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i)]$

NAIVE BAYES CLASSIFIER

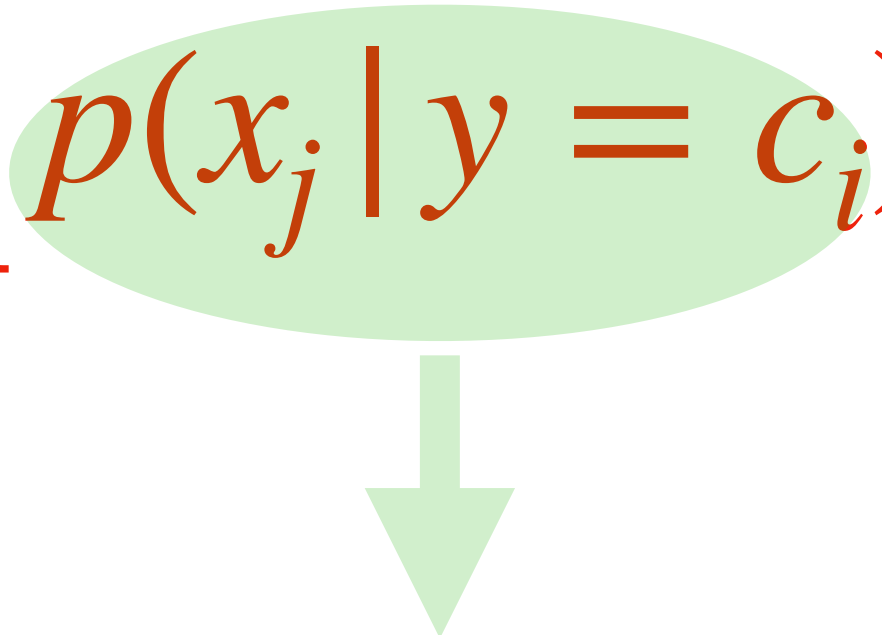
We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i)]$



Simply calculate
from training data

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i)]$



Easy if all features
qualitative: calculate
probability by checking
training observations
with $y = c_i$.

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i) \right]$

Ex:

Day	Outlook	Temp	Humidity	Beach?
1	Sunny	High	High	Yes
2	Sunny	High	Normal	Yes
3	Sunny	Low	Normal	No
4	Sunny	Mild	High	Yes
5	Rain	Mild	Normal	No
6	Rain	High	High	No
7	Rain	Low	Normal	No
8	Cloudy	High	High	No
9	Cloudy	High	Normal	Yes
10	Cloudy	Mild	Normal	No

Let $X = \langle \text{sunny, mild, high} \rangle$
Beach or no beach?

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i) \right]$

Ex:

Day	Outlook	Temp	Humidity	Beach?
1	Sunny	High	High	Yes
2	Sunny	High	Normal	Yes
3	Sunny	Low	Normal	No
4	Sunny	Mild	High	Yes
5	Rain	Mild	Normal	No
6	Rain	High	High	No
7	Rain	Low	Normal	No
8	Cloudy	High	High	No
9	Cloudy	High	Normal	Yes
10	Cloudy	Mild	Normal	No

Let $X = \langle \text{sunny, mild, high} \rangle$
Beach or no beach?

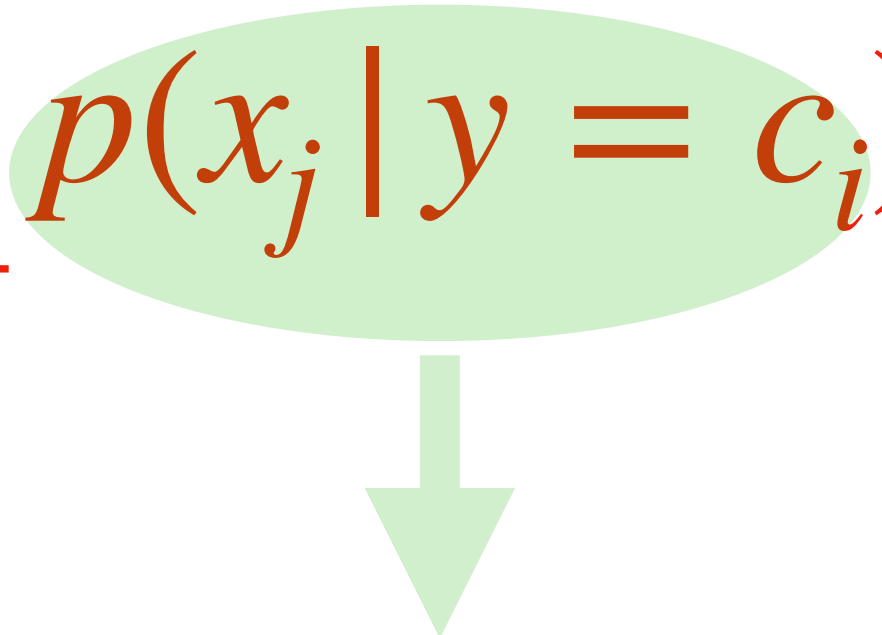
Note: Computed values are not probabilities since denominator is omitted.

$$\begin{aligned} p(\text{beach} | \text{sunny, mild, high}) &= p(\text{beach})p(\text{sunny} | \text{beach})p(\text{mild} | \text{beach})p(\text{high} | \text{beach}) \\ &= 4/10 \times 3/4 \times 1/4 \times 2/4 = 0.0375 \end{aligned}$$

$$\begin{aligned} p(\text{nobeach} | \text{sunny, mild, high}) &= p(\text{no})p(\text{sunny} | \text{no})p(\text{mild} | \text{no})p(\text{high} | \text{no}) \\ &= 6/10 \times 1/6 \times 2/6 \times 2/6 = 0.0111 \end{aligned}$$

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i)]$



Need to generalize it to a probability density function if quantitative predictor.

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i)]$

Can further transform with $\log$ for numerical stability:

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [\log p(y = c_i) + \sum_{j=1}^m \log p(x_j | y = c_i)]$$

NAIVE BAYES CLASSIFIER

We have the class of X : $\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[p(y = c_i) \prod_{j=1}^m p(x_j | y = c_i) \right]$

Can further transform with $\log$ for numerical stability:

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} \left[\log p(y = c_i) + \sum_{j=1}^m \log p(x_j | y = c_i) \right]$$

For fitting the model: Maximize likelihood,

$$\arg \max_{\forall \theta} \left[\sum_{r=1}^n \log p(y^{(r)} = c_i) + \sum_{r=1}^n \sum_{j=1}^m \log p(x_j^{(r)} | y^{(r)} = c_i) \right]$$

Under a specific parametrization θ_j of feature j

NAIVE BAYES CLASSIFIER

```
class sklearn.naive_bayes.GaussianNB(*, priors=None, var_smoothing=1e-09)
```

GaussianNB: Numeric Features assumed to be Gaussian.

$$p(x_j | y = c_i) \sim N(\mu_{ji}, \sigma_{ji}^2)$$

Filter with respect to c_i and solve for:

μ_{ji}, σ_{ji} via MLE as part of θ_j .

Example dataset:

Petal Length	Petal Width	Class
4.5	3.0	Setosa
6.0	2.7	Virginica
...	...	...

NAIVE BAYES CLASSIFIER

```
class sklearn.naive_bayes.MultinomialNB(*, alpha=1.0,  
force_alpha=True, fit_prior=True, class_prior=None)
```

MultinomialNP: Discrete features, like word counts in text classification

← Features (Word-topic) →

Crowds	State	Referee	...	Basketball
25	3	18	...	1
7	0		...	?

Example dataset

NAIVE BAYES CLASSIFIER

```
class sklearn.naive_bayes.MultinomialNB(*, alpha=1.0,  
force_alpha=True, fit_prior=True, class_prior=None)
```

MultinomialNP: Discrete features, like word counts in text classification

← Features (Word-topic) →

Crowds	State	Referee	...	Basketball
25	3	18	...	1
7	0		...	?

Example dataset

If mixed features: Binning (numeric features) followed by Multinomial
OR

Independently fit one Gaussian and one multinomial, then fit on two numeric features (class probabilities) using Gaussian.

DISTANCE-BASED METHODS

Distance metrics: A distance measure is a **metric** if,

- Positivity: $d(x, y) \geq 0 \quad \forall x, y$
- Identity: $d(x, y) = 0$ iff $x = y$
- Symmetry: $d(x, y) = d(y, x) \quad \forall x, y$
- Triangular inequality: $d(x, y) \leq d(x, z) + d(z, y) \quad \forall x, y, z$

DISTANCE-BASED METHODS

Distance metrics: A distance measure is a **metric** if,

- Positivity: $d(x, y) \geq 0 \quad \forall x, y$
- Identity: $d(x, y) = 0$ iff $x = y$
- Symmetry: $d(x, y) = d(y, x) \quad \forall x, y$
- Triangular inequality: $d(x, y) \leq d(x, z) + d(z, y) \quad \forall x, y, z$

Which ones are metrics?

Euclidean distance, correlation coefficient, cosine similarity/dot product, cheapest airfare.

DISTANCE-BASED METHODS

L_r distance metric (norm): Minkowski distance

Two points in m -dimensional space:

$p = (p_1, p_2, \dots, p_m)$, $q = (q_1, q_2, \dots, q_m)$

$$d_r(p, q) = \left(\sum_{i=1}^m |p_i - q_i|^r \right)^{1/r}$$

DISTANCE-BASED METHODS

L_r distance metric (norm): Minkowski distance

Two points in m -dimensional space:

$$p = (p_1, p_2, \dots, p_m), q = (q_1, q_2, \dots, q_m) \quad d_r(p, q) = \left(\sum_{i=1}^m |p_i - q_i|^r \right)^{1/r}$$

Ex: Consider $m = 2$. Let $p = (p_1, p_2)$, $q = (q_1, q_2)$:

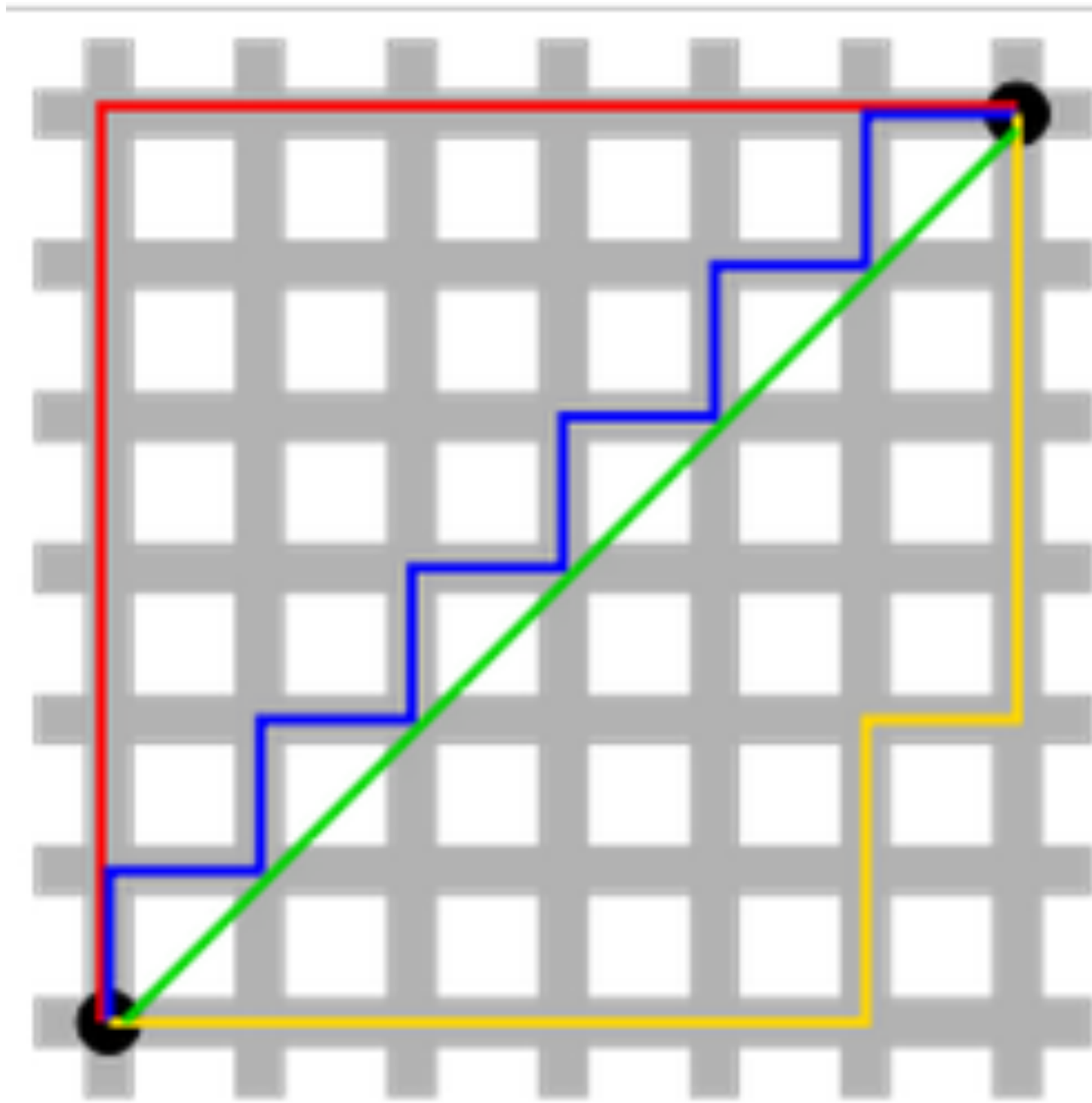
(Manhattan distance) $d_1(p, q) = |p_1 - q_1| + |p_2 - q_2|$

(Euclidean distance) $d_2(p, q) = \sqrt{|p_1 - q_1|^2 + |p_2 - q_2|^2}$

(Chebyshev distance) $d_\infty(p, q) = \max(|p_1 - q_1|, |p_2 - q_2|)$

DISTANCE-BASED METHODS

Some Properties of L_r distance:

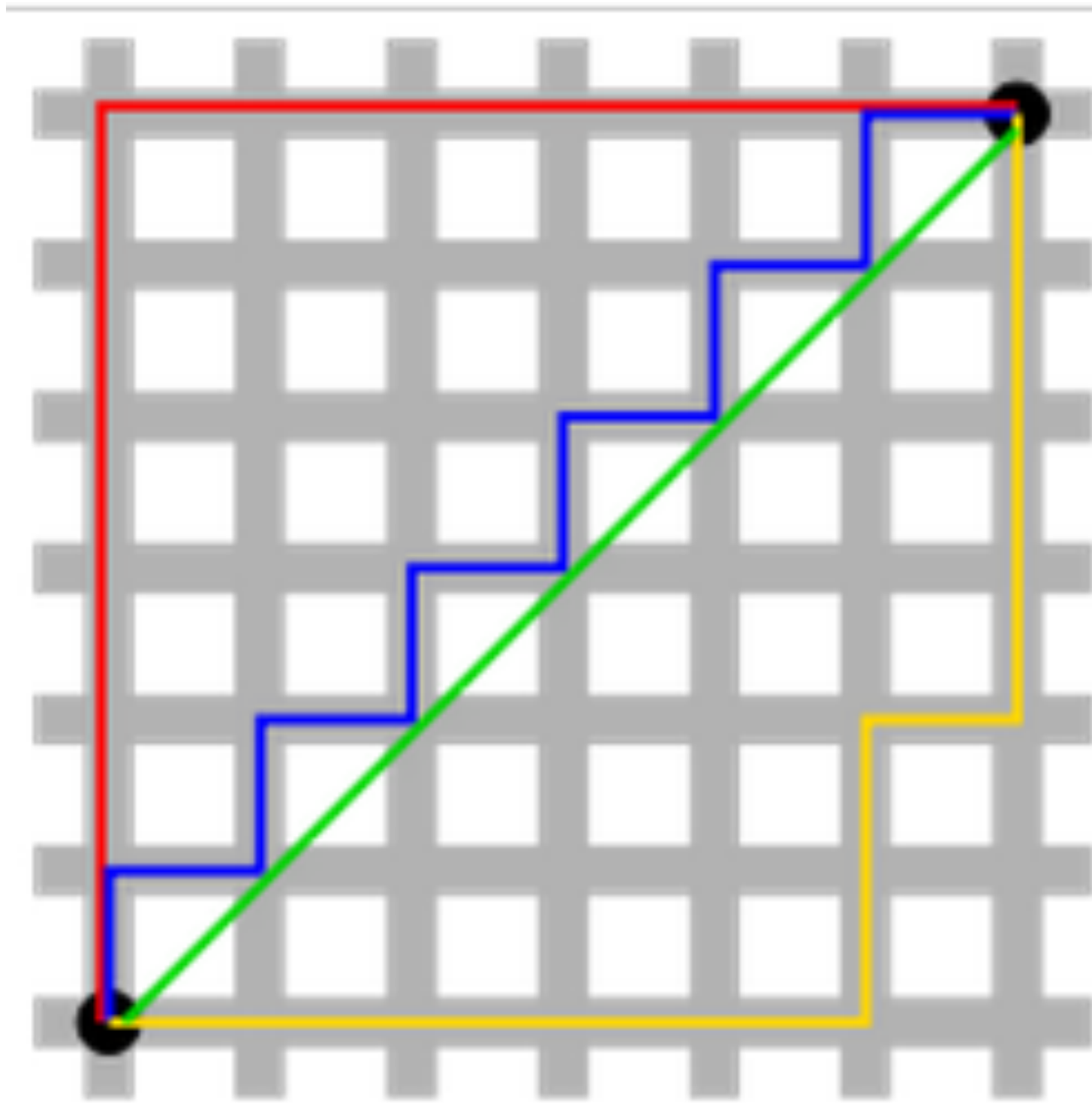


What is the shortest path between the two black points in L_1 ?

What is the shortest path between the two black points in L_2 ?

DISTANCE-BASED METHODS

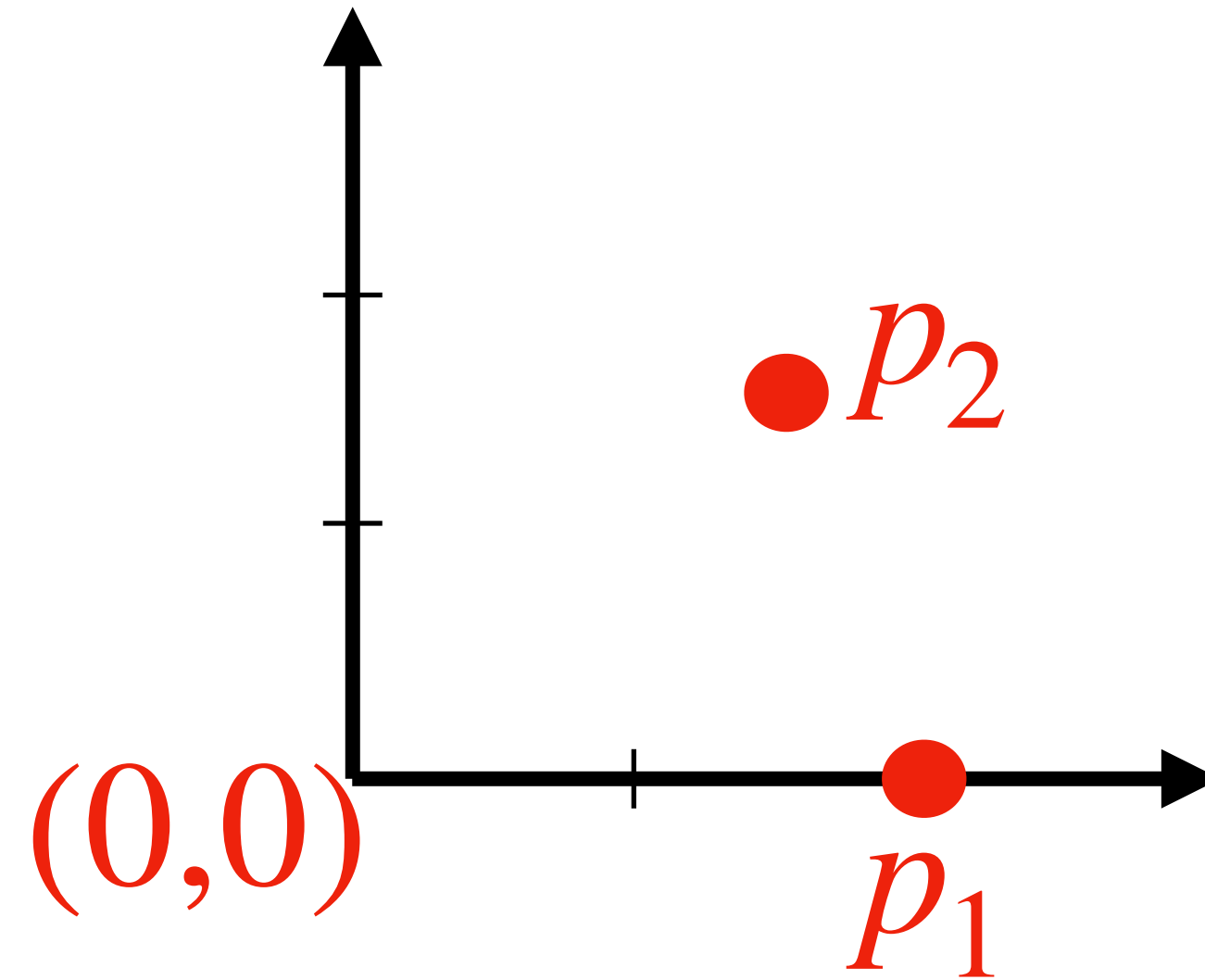
Some Properties of L_r distance:



- Many shortest paths in L_1
- r regulates trade off between largest and total dimensional difference:
 - L_1 : all dimensions contribute in the amount of difference.
 - L_2 : dimensions with larger difference contribute more.
 - L_∞ : only the dimension with largest difference contributes.
- Must normalize features/dimensions.

DISTANCE-BASED METHODS

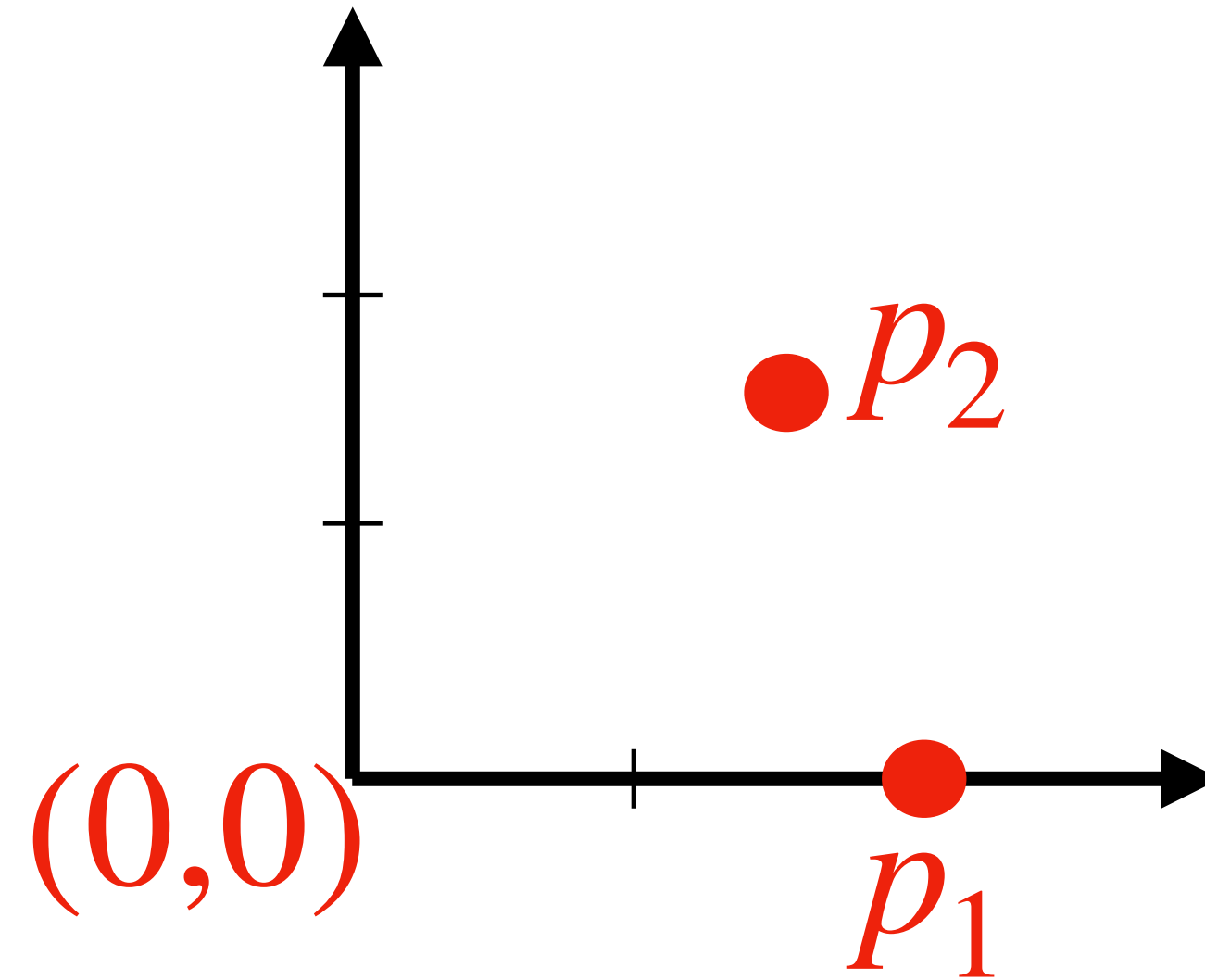
Ex: Let $p_1 = (2,0)$ and $p_2 = (1.5,1.5)$.



Which point is farther from origin $(0,0)$ in L_1, L_2, L_∞ ?

DISTANCE-BASED METHODS

Ex: Let $p_1 = (2,0)$ and $p_2 = (1.5,1.5)$.



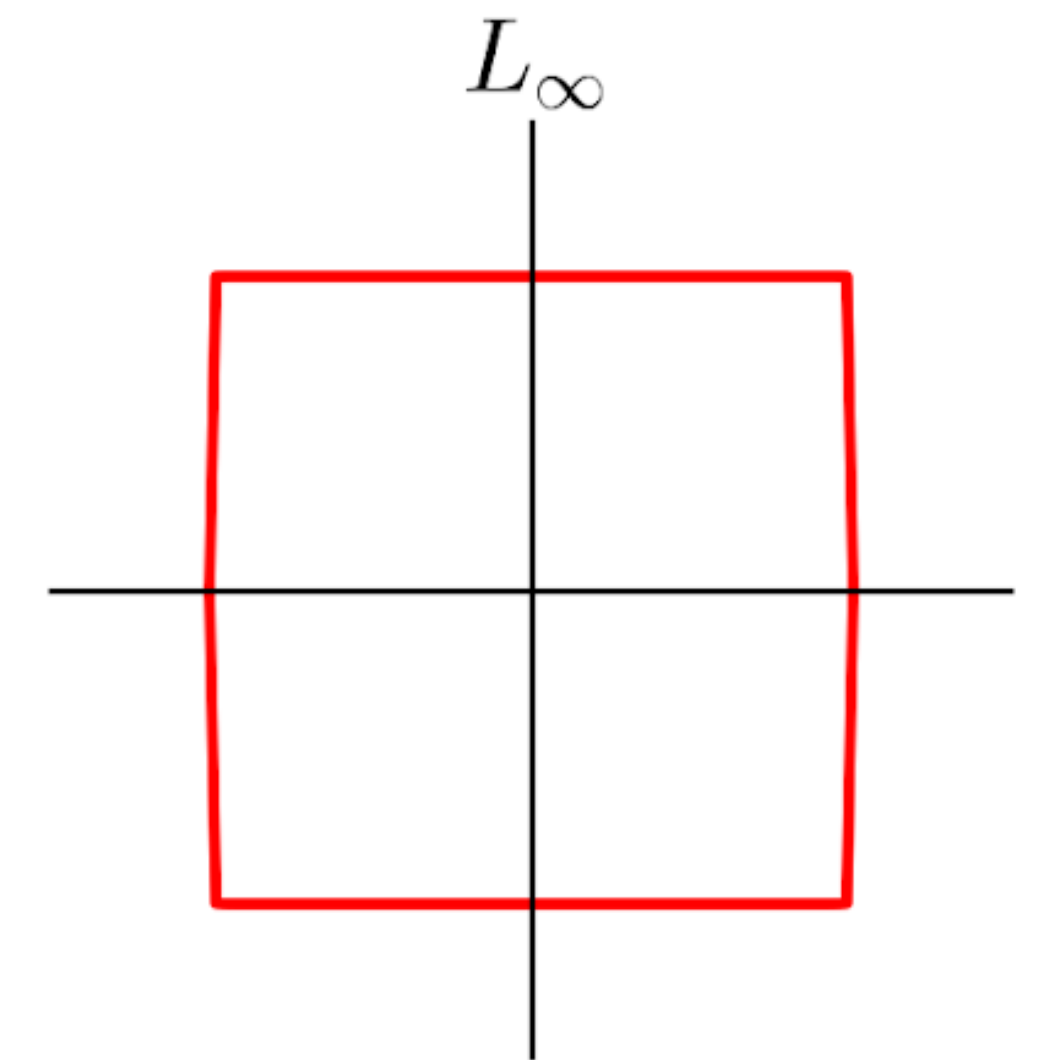
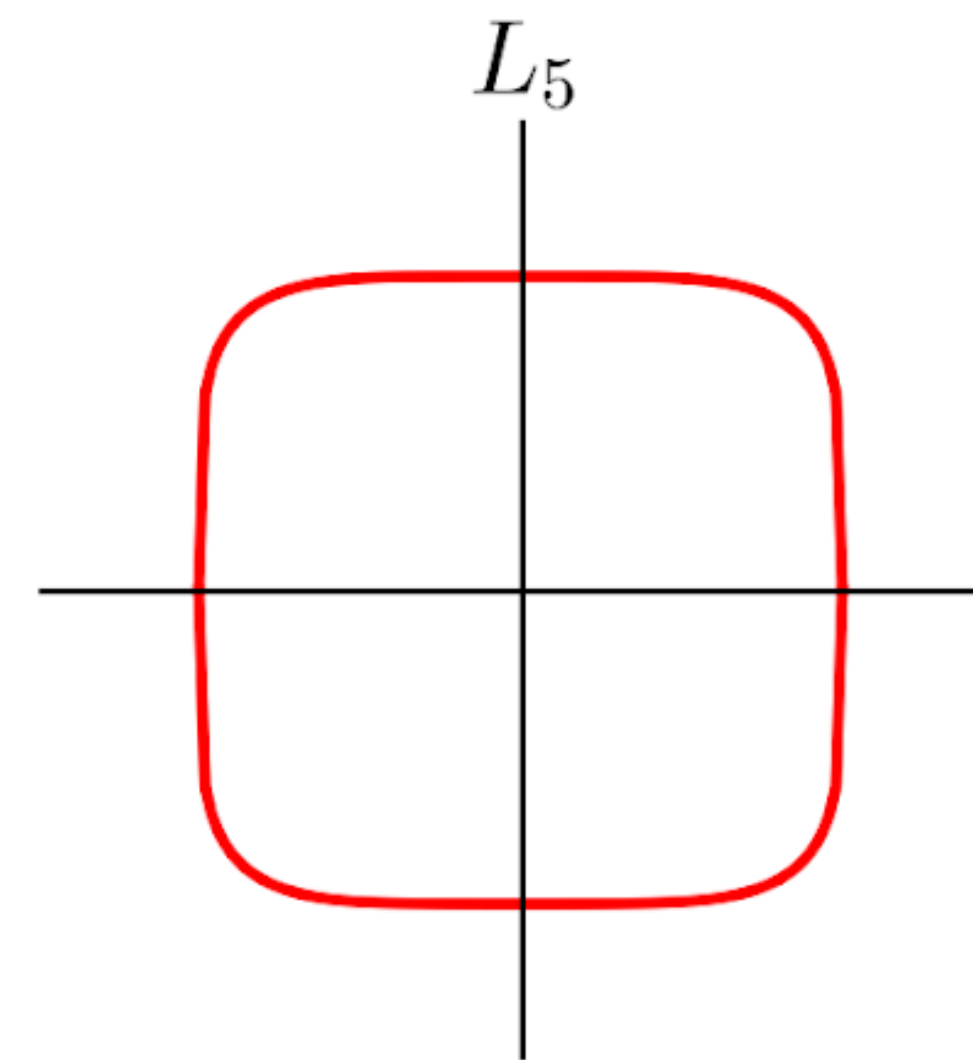
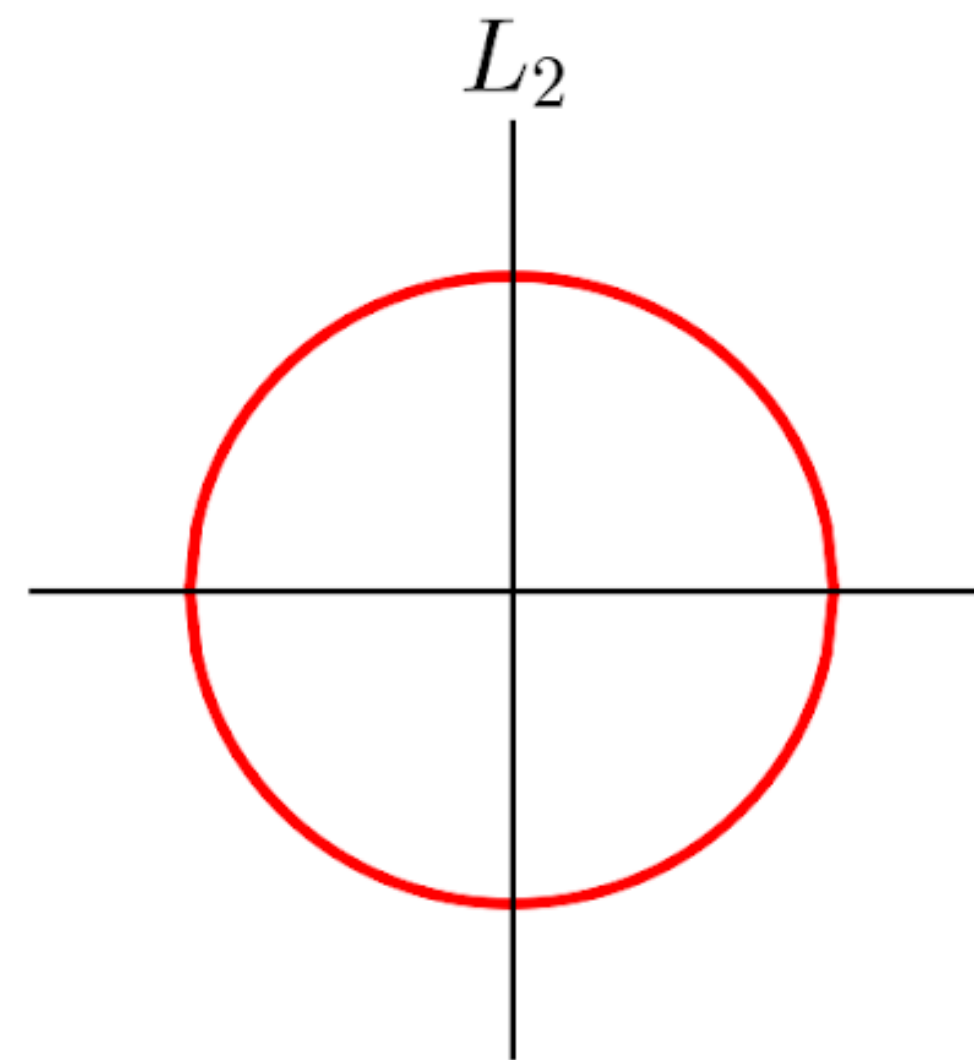
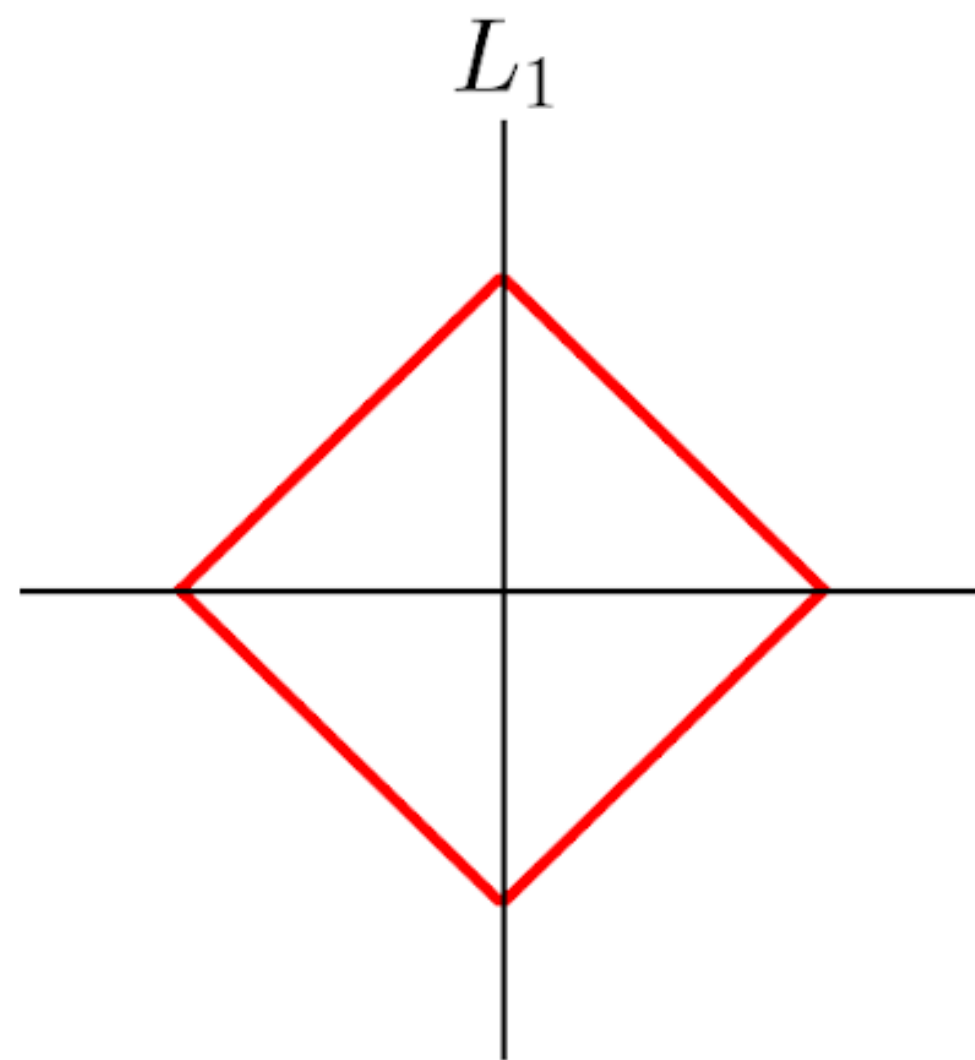
Which point is farther from origin $(0,0)$ in L_1, L_2, L_∞ ?

- In L_1 distances are 2, 3 respectively. p_2 is farther.
- In L_2 distances are 2, 2.12 respectively. p_2 is farther.
- In L_∞ distances are 2, 1.5 respectively. p_1 is farther.

DISTANCE-BASED METHODS

Circles of equidistant points in different L_r :

(We have already seen the first two when discussing regularization)



DISTANCE-BASED METHODS

Other types of distances: Distance between **vectors**

	Features (Word-topic)				Target
	Crowds	State	Referee	...	Basketball
Doc_1	25	3	18	...	1
Doc_2	7	0	4	...	?

DISTANCE-BASED METHODS

Other types of distances: Distance between **vectors**

- Doc_1 long, Doc_2 short
- Both can be about basketball, though their point wise distances large

	Features (Word-topic)				Target
	Crowds	State	Referee	...	Basketball
Doc_1	25	3	18	...	1
Doc_2	7	0	4	...	?

DISTANCE-BASED METHODS

Other types of distances: Distance between **vectors**

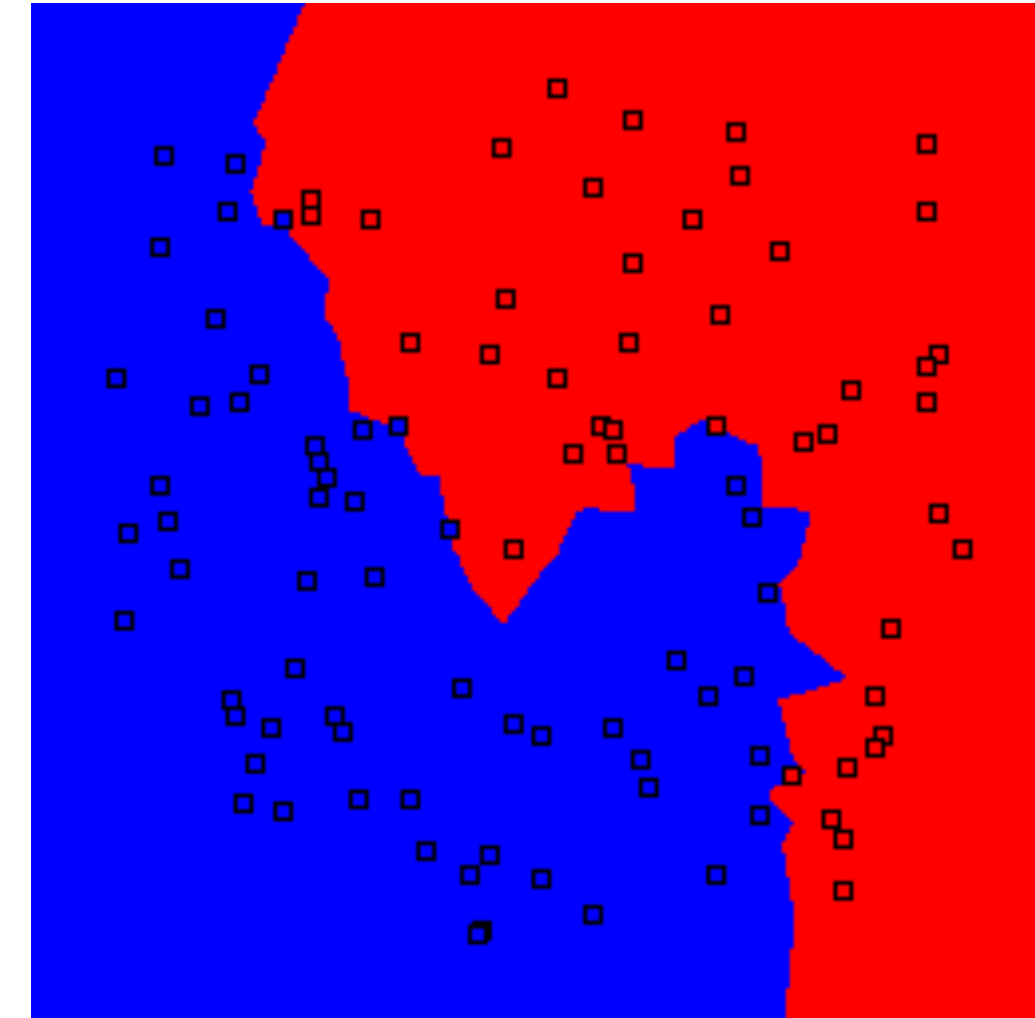
- Doc_1 long, Doc_2 short
- Both can be about basketball, though their point wise distances large
- Cosine similarity-based distance: $1 - \text{Cos}(p, q)$
- Not a proper metric. Instead, use angular distance:
 $\text{Arccos}(\text{Cos}(p, q)) / \pi$

	Features (Word-topic)				Target
	Crowds	State	Referee	...	Basketball
Doc_1	25	3	18	...	1
Doc_2	7	0	4	...	?

NEAREST NEIGHBOR CLASSIFICATION

Idea:

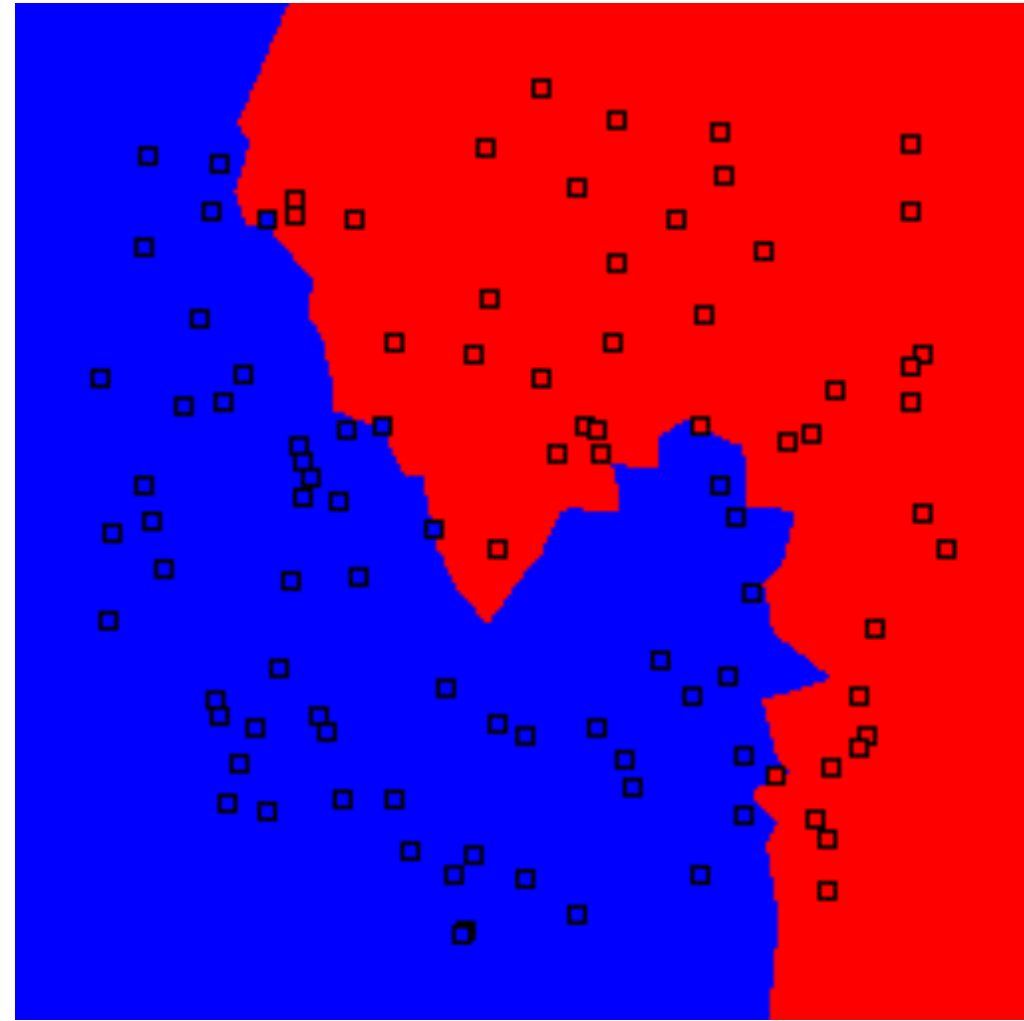
Given test point p , find training points with smallest *distance* to p , assign a class based on their classes.



NEAREST NEIGHBOR CLASSIFICATION

Idea:

Given test point p , find training points with smallest *distance* to p , assign a class based on their classes.



Advantages:

- Simplicity
- Interpretability
- Nonlinearity



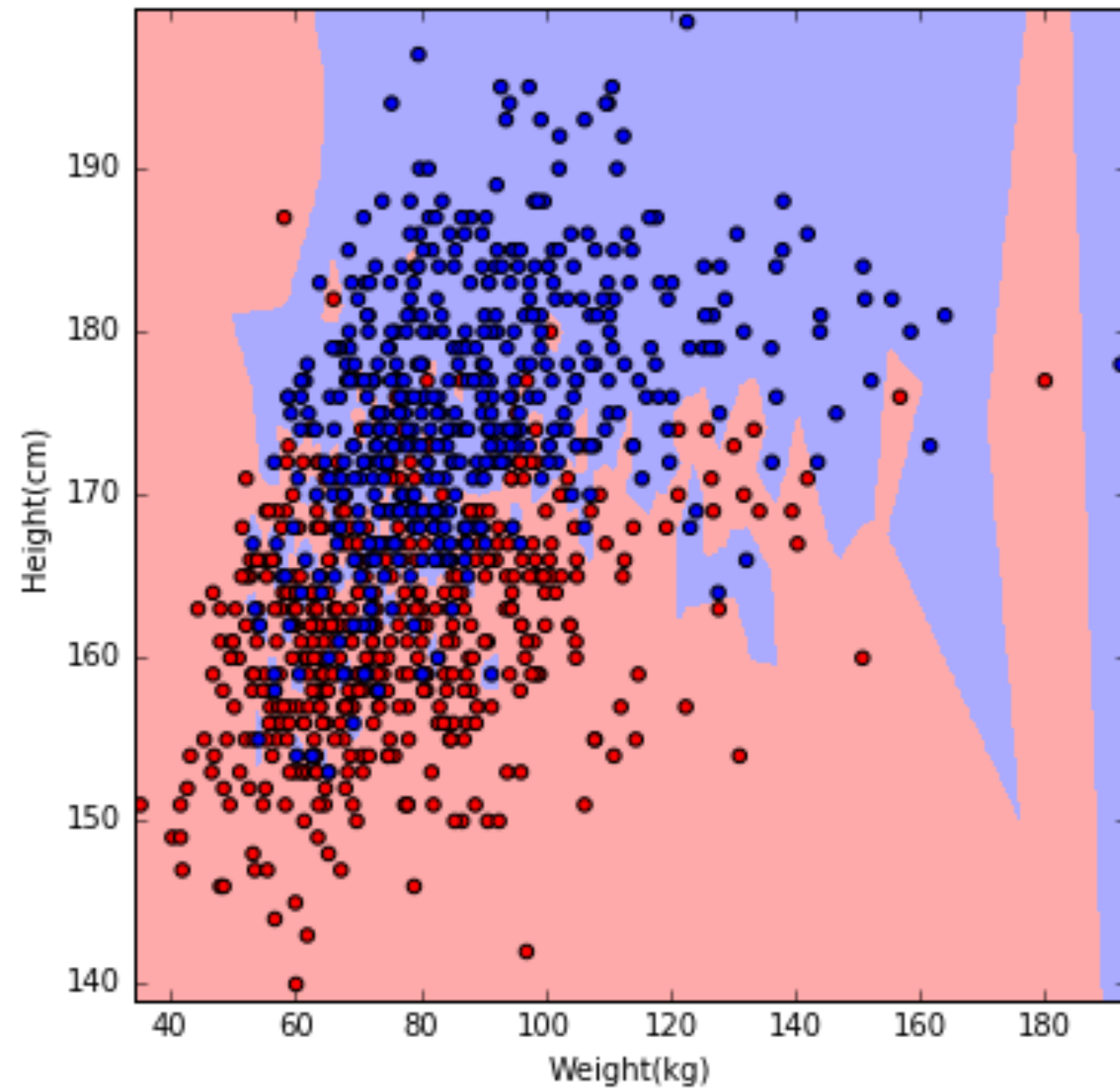
k -nearest neighbors:

Take a majority vote of the k -nearest neighbor training points.

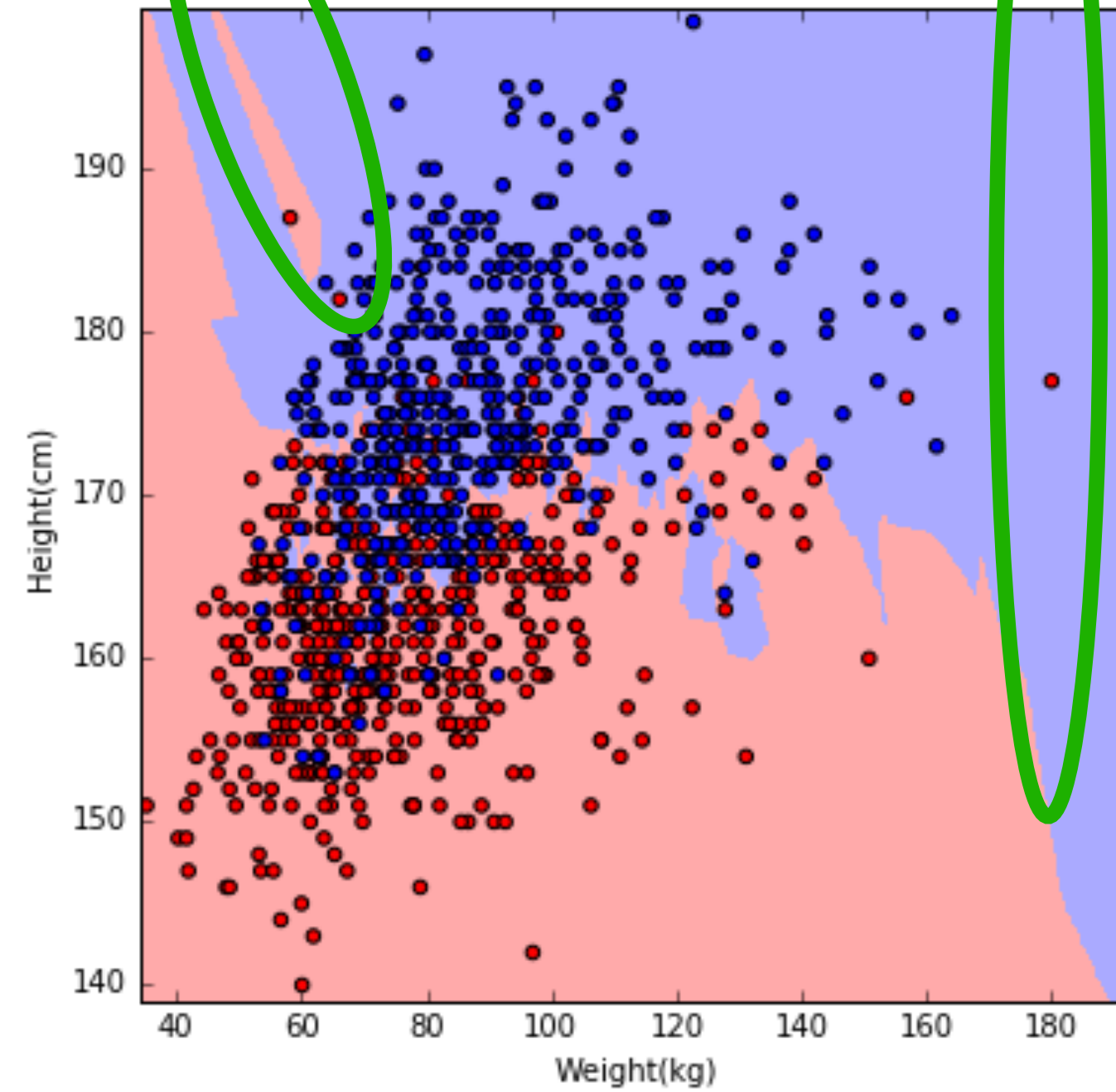
NEAREST NEIGHBOR CLASSIFICATION

The effect of k :

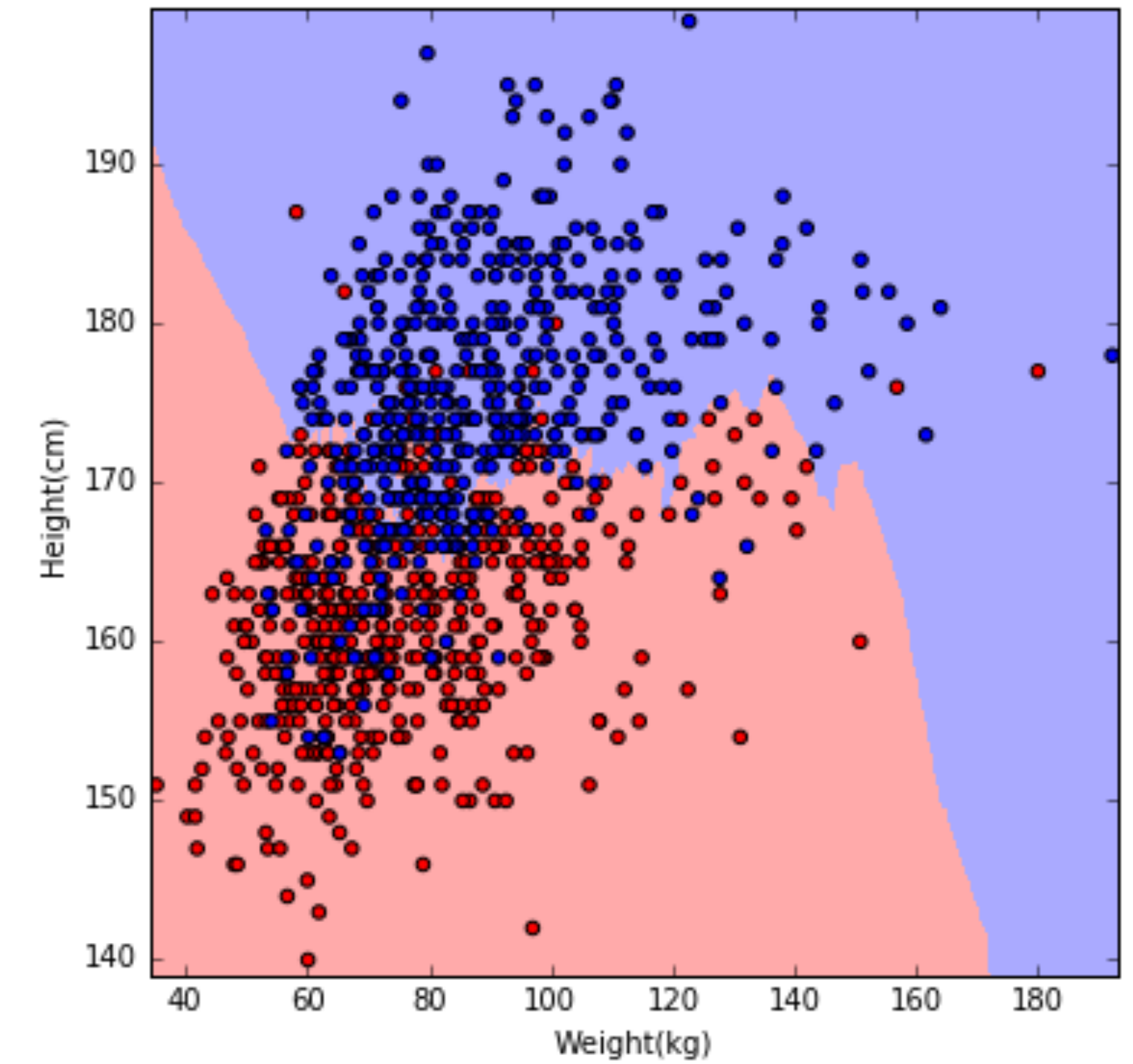
$k = 1$



$k = 3$



$k = 10$



- Large $k \Rightarrow$ Smoother boundary
- To determine a good value of k we can do cross validation.

NEAREST NEIGHBOR CLASSIFICATION

`sklearn.neighbors.KNeighborsClassifier`

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30,  
p=2, metric='minkowski', metric_params=None, n_jobs=None) \[source\]
```

`n_neighbors` is our k .

`weights='uniform'` $\Rightarrow$ Each point in a neighborhood weighted equally

`weights='distance'` $\Rightarrow$ Each point in a neighborhood weighted by inverse distance.

If $p = 2$ we want L_2 distance.